

Experiment 02

Multiple Linear Regression, Ridge Regression, and Lasso Regression

1. Dataset Source

Dataset Name: Insurance Premium Prediction Dataset

Source: Kaggle

Link: <https://www.kaggle.com/datasets/noordeen/insurance-premium-prediction/data>

This dataset is used to predict medical insurance expenses based on demographic and lifestyle attributes.

2. Dataset Description

The dataset contains 1338 records with the following features:

Input Features:

- age – Age of the individual
- sex – Gender (male/female)
- bmi – Body Mass Index
- children – Number of dependents
- smoker – Smoking status (yes/no)
- region – Residential region (northeast, northwest, southeast, southwest)

Target Variable:

- expenses – Annual medical insurance cost (continuous variable)

Dataset Characteristics:

- Mix of numerical and categorical features
- Moderate dataset size
- Suitable for regression analysis
- No severe missing values

The objective is to predict insurance expenses using multiple regression techniques.

3. Mathematical Formulation of Algorithms

3.1 Multiple Linear Regression

The regression model is defined as:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Where:

- y = target variable (expenses)
- x_i = independent variables
- β_i = regression coefficients
- ϵ = error term

The objective is to minimize the Residual Sum of Squares (RSS):

$$RSS = \sum (y_i - \hat{y}_i)^2$$

3.2 Ridge Regression (L2 Regularization)

Ridge adds a penalty term to prevent overfitting:

$$Loss = RSS + \lambda \sum \beta_i^2$$

- λ controls regularization strength
- Shrinks coefficients but does not eliminate them

3.3 Lasso Regression (L1 Regularization)

Lasso adds an absolute penalty term:

$$Loss = RSS + \lambda \sum |\beta_i|$$

- Can shrink some coefficients exactly to zero
- Performs feature selection

4. Algorithm Limitations

Linear Regression

- Assumes linear relationship between variables
- Sensitive to multicollinearity

- Sensitive to outliers

Ridge Regression

- Does not perform feature selection
- Requires tuning of hyperparameter λ

Lasso Regression

- Can remove important correlated features
- May underperform when strong multicollinearity exists

5. Methodology / Workflow

Step 1: Data Loading

- Dataset imported using Pandas

```
[1]
✓ 2s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
```

```
[2]
✓ 0s
df = pd.read_csv("insurance.csv")

print("Dataset Shape:", df.shape)
df.head()
```

Dataset Shape: (1338, 7)

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86

Step 2: Data Preprocessing

- One-Hot Encoding applied to categorical variables
- Features scaled using StandardScaler

```
[6] df_encoded = pd.get_dummies(df, drop_first=True)
    print("Encoded Dataset Shape:", df_encoded.shape)
    df_encoded.head()
```

... Encoded Dataset Shape: (1338, 9)

	age	bmi	children	expenses	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.9	0	16884.92	False	True	False	False	True
1	18	33.8	1	1725.55	True	False	False	True	False
2	28	33.0	3	4449.46	True	False	False	True	False
3	33	22.7	0	21984.47	True	False	True	False	False
4	32	28.9	0	3866.86	True	False	True	False	False

```
[9] X = df_encoded.drop("expenses", axis=1)
    y = df_encoded["expenses"]

    print("Feature Shape:", X.shape)
    print("Target Shape:", y.shape)
```

Feature Shape: (1338, 8)
Target Shape: (1338,)

```
[10] from sklearn.preprocessing import StandardScaler

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    print("Scaled Feature Shape:", X_scaled.shape)
```

Scaled Feature Shape: (1338, 8)

Step 3: Train-Test Split

```
[11] from sklearn.model_selection import train_test_split

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y,
        test_size=0.2,
        random_state=42
    )

    print("Training Shape:", X_train.shape)
    print("Testing Shape:", X_test.shape)
```

... Training Shape: (1070, 8)
Testing Shape: (268, 8)

Step 4: Model Training

- Multiple Linear Regression
- Ridge Regression (alpha = 1.0)
- Lasso Regression (alpha = 0.1)

```
[12]
✓ Os
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Create model
linear_model = LinearRegression()

# Train model
linear_model.fit(X_train, y_train)

# Predict
y_pred_linear = linear_model.predict(X_test)

# Evaluation
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print("Linear Regression Results")
print("MSE:", mse_linear)
print("R2 Score:", r2_linear)
```

Linear Regression Results
MSE: 33600065.35507784
R2 Score: 0.7835726930039905

```
[13]
✓ Os
from sklearn.linear_model import Ridge

# Create Ridge model
ridge_model = Ridge(alpha=1.0)

# Train
ridge_model.fit(X_train, y_train)

# Predict
y_pred_ridge = ridge_model.predict(X_test)

# Evaluation
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print("Ridge Regression Results")
print("MSE:", mse_ridge)
print("R2 Score:", r2_ridge)
```

Ridge Regression Results
MSE: 33608088.75005093
R2 Score: 0.7835210120995446

```

14] from sklearn.linear_model import Lasso
    ' Os

# Create Lasso model
lasso_model = Lasso(alpha=0.1)

# Train
lasso_model.fit(X_train, y_train)

# Predict
y_pred_lasso = lasso_model.predict(X_test)

# Evaluation
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print("Lasso Regression Results")
print("MSE:", mse_lasso)
print("R2 Score:", r2_lasso)

```

✓ Lasso Regression Results
MSE: 33600486.839937285
R2 Score: 0.7835699781035251

Step 5: Model Evaluation

- Mean Squared Error (MSE)
- R² Score

```

15 print("MODEL COMPARISON")
    print("-----")
    print("Linear -> R2:", r2_linear)
    print("Ridge -> R2:", r2_ridge)
    print("Lasso -> R2:", r2_lasso)

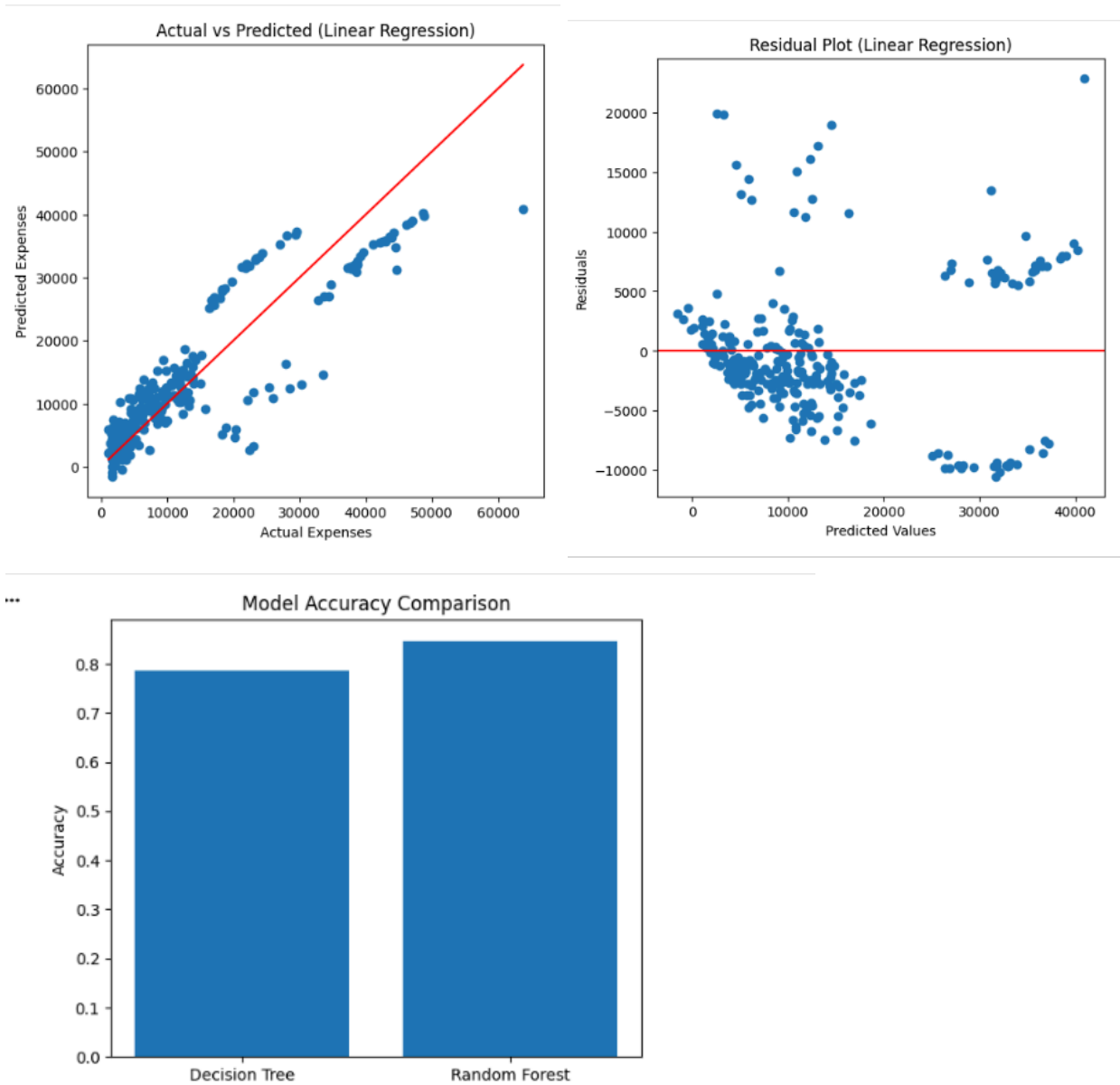
```

MODEL COMPARISON

Linear -> R2: 0.7835726930039905
Ridge -> R2: 0.7835210120995446
Lasso -> R2: 0.7835699781035251

Step 6: Visualization

- Actual vs Predicted Plot
- Residual Plot
- Model Comparison Bar Chart



6. Performance Analysis

R² Scores Obtained:

- Linear Regression: 0.78357
- Ridge Regression: 0.78352
- Lasso Regression: 0.78356

Interpretation:

- All three models performed almost identically.

- The models explain approximately 78.3% of the variance in insurance expenses.
- Regularization did not significantly improve performance.
- Indicates minimal overfitting and limited multicollinearity.

The dataset appears well-suited for linear modeling.

7. Hyperparameter Tuning

Hyperparameter tuning can be performed using GridSearchCV.

Example for Ridge:

$\alpha=[0.01,0.1,1,10,100]$

Example for Lasso:

$\alpha=[0.001,0.01,0.1,1]$

Observations:

- Optimal alpha values did not significantly change R^2 .
- Confirms dataset stability and low overfitting risk.
- Linear model already performs efficiently.

8. Final Conclusion

- Multiple Linear Regression provides strong predictive performance.
- Ridge and Lasso offer regularization benefits but did not significantly improve accuracy in this dataset.
- The model explains a substantial portion of variability in insurance expenses.
- Ensemble or nonlinear models may further improve performance if required.